

Nicole Go

Professor Minnes Kemp

CSE 105

Project Task 2

22 February, 2023

Project Task 1

Start with your face and your student ID for a few seconds at the beginning, and introduce yourself audibly while on screen. You don't have to be on camera for the rest of the video, though it's fine if you are. We are looking for a brief confirmation that it's you creating the video and doing the work you submitted.

- Present the NFA you will be working with. Your video should include a clear and precise explanation of why the language of this NFA is not empty and also not the set of all strings over Σ .

- Show on the screen and explain the code for your program, including the software design choices you made (e.g. which data structures are you using, etc.)

- and any resources you

Used.

- The video should clearly describe which programming language was chosen for the implementation and gives the reasons why.

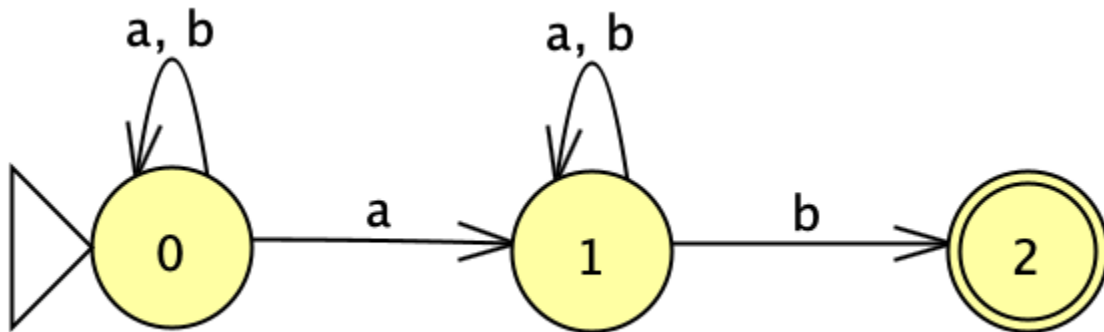
- Show on the screen and explain the representation of the NFA that you will input to the program.

- Demonstrate running your code on your example input. The video should include screencasts of running the code live. Explain why the output of your program is what you would expect,

- by connecting the output of the program to a DFA and discussing which strings are accepted / rejected by this DFA.

Clear specification of alphabet and state diagram of chosen three-state NFA.

The alphabet for my example three-state NFA is $\{a, b\}$ and its state diagram is taken from the Week 3 Wednesday lecture:



Documentation for program converting NFA to DFA: include a description of how NFA and DFA are represented in the program and give instructions for running it.

```

1
2 # CHATGPT: "how can i fix this code to make a powerset? how can i make it so there are no duplicates?"
3 # CHATGPT: "why are there commas after my single items in tuples?"
4 # CHATGPT: "what is the difference between split and replace?"
5 # CHATGPT: "what is wrong with my for loop and if statements for my transition function?"
6 # CHATGPT: "how can i remove duplicates from a list?"
7
8 def main():
9     NFAToDFA()
10
11 def NFAToDFA():
12     user_input_NFA = input("Please input a NFA by it's formal definition (Q, I, δ, q*, F) and separating commas for each component. Please define the transition function as a Python list of transitions for each state and input symbol.")
13     split_after_transition = user_input_NFA.split(",")
14     components_before_start = split_after_transition[0].split(",")
15     components_after_transition = split_after_transition[1].split(",")
16
17     states_NFA = list(components_before_start[0].split(","))
18     states_NFA[0] = states_NFA[0].strip("{}")
19     for i in range(len(states_NFA)):
20         states_NFA[i] = states_NFA[i].strip()
21
22     # alphabet would be same because DFA consumes the same input
23     alphabet = list(components_before_start[1].split(","))
24     alphabet[0] = alphabet[0].strip("{}")
25     for i in range(len(alphabet)):
26         alphabet[i] = alphabet[i].strip()
27
28     # converts transition function into the form of a dictionary where a tuple of the state and the input is the key, the power set (as a list) of possible states to go to is the value
29
30     transition_function_NFA = list(components_after_transition[2].split(","))
31     transition_function_NFA[0] = transition_function_NFA[0].strip("{}")
32     transition_function_dict_NFA = {}
33     for item in transition_function_NFA:
34         item = item.strip()
35         item = item.replace("6", "")
36         item = item.replace(")", "")
37         item = item.replace("(", "")
38         item = item.replace("'", "")
39         item = item.replace(" ", "")
40         split_input_output = item.split(" ")
41         if len(split_input_output) == 2:
42             input_state_input = split_input_output[0].split(",")
43             if len(split_input_output) == 2:
44                 tuple_state_and_input = tuple(input_state_input)
45                 list_output_powerset_states = split_input_output[1].split(",")
46                 transition_function_dict_NFA[tuple_state_and_input] = list_output_powerset_states
47             else:
48                 print("Invalid format")
49         else:
50             print("Invalid format")
51
52     # start state would be the same in DFA assuming there are no spontaneous moves

```

```

# start state would be the same in DFA assuming there are no spontaneous moves
# (because there would then be no other states accessible from the start state that don't require consuming an input)
start_state = components_after_transition[0]
start_state = start_state.strip()

accept_states_NFA = components_after_transition[1]
accept_states_NFA = accept_states_NFA.replace(")", "")
accept_states_NFA = accept_states_NFA.replace("(", "")
accept_states_NFA = accept_states_NFA.replace("'", "")
accept_states_NFA = accept_states_NFA.replace(" ", "")
accept_states_NFA_list = accept_states_NFA.split(",")
if len(accept_states_NFA_list) > 1:
    for i in range(len(accept_states_NFA_list)):
        accept_states_NFA[i] = accept_states_NFA[i].strip()
        accept_states_NFA[i] = accept_states_NFA[i].strip("{}")

# in the form of a list creates powerset of states_NFA (so all subsets of the set of states in NFA)
# where each element is a Python tuple representing the subset/state

states_DFA = []
states_DFA.append(tuple())
for item in states_NFA:
    states_DFA.append(tuple(item))
for i in range(1 <= len(states_NFA)):
    subset = []
    for j in range(len(states_NFA)):
        if i & (1 <= j):
            subset.append(states_NFA[j])
    states_DFA.append(tuple(subset))
states_DFA = list(set(states_DFA))
states_DFA.sort()

# creates dictionary of transitions in DFA form Q x I -> Q where Q is a state of the DFA (where states are subsets) and every input has an arrow
# the keys are the input (as tuples) and the values are the output (a single tuple element of states_DFA)
# empty set state points to itself

transition_function_dict_DFA = {}
for symbol in alphabet:
    transition_function_dict_DFA[(states_DFA[0], symbol)] = states_DFA[0]
for state in states_DFA:
    for symbol in alphabet:
        list_accepted_from_multiple_state = []
        for item in state:
            if (item, symbol) in transition_function_dict_NFA:
                output_string = str(transition_function_dict_NFA[(item, symbol)])
                final_output_string = output_string.replace(' ', '').replace('(', '').replace(')', '').replace("'", '')
                final_output_string = final_output_string.strip("{}")

```

In the form that my program takes (formal definition), this NFA would be written as:

$(\{0, 1, 2\}, \{a, b\}, [\delta(0, a) = \{0, 1\}, \delta(0, b) = \{0\}, \delta(1, a) = \{1\}, \delta(1, b) = \{1, 2\}], 0, \{2\})$. This is where the set of states, alphabet symbols, and accept states are all normal set notation within $\{ \}$.

The start state is just the name of the state and the transition function is a list of transitions in Python style surrounded by $[]$. All components are in a comma-separated list and are in one line of input. The list is surrounded by parentheses. DFAs are separated and organized by new lines.

Each component is labelled and the transition function has a new line for each transition to make it easier for the reader to understand. To run the program, if in the directory that has the file, we can run “python3 ./cse105projecttask1.py” on the terminal and put in the input of whatever NFA we want.

Printout of code for program converting NFA to DFA.

```
# CHATGPT: "how can i fix this code to make a powerset? how can i make it so there are no
duplicates?"
```

```
# CHATGPT: "why are there commas after my single items in tuples?"
```

```
# CHATGPT: "what is the difference between split and replace?"
```

```
# CHATGPT: "what is wrong with my for loop and if statements for my transition function?"
```

```
# CHATGPT: "how can i remove duplicates from a list?"
```

```
def main():
```

```
    NFAtoDFA()
```

```
def NFAtoDFA():
```

```
    user_input_NFA = input("Please input a NFA by it's formal definition (Q, Σ, δ, q0, F) and
separating commas for each component. Please define the transition function as a Python list of
transitions for each combination of state and input (δ(q, a) = P(Q)) \n Ex. ({q0, q1, q2}, {a, b},
[δ(q0, a) = {q1, q2}, δ(q1, b) = {q2}], q0, {q1, q2}): \n")

    split_after_transition = user_input_NFA.split("},")
```

```

components_before_start = split_after_transition[0].split("}, ", 2)
components_after_transition = split_after_transition[1].split(", ", 1)

states_NFA = list(components_before_start[0].split(","))
states_NFA[0] = states_NFA[0].strip("{")

for i in range(len(states_NFA)):
    states_NFA[i] = states_NFA[i].strip()

# alphabet would be same because DFA consumes the same input
alphabet = list(components_before_start[1].split(","))
alphabet[0] = alphabet[0].strip("{")

for i in range(len(alphabet)):
    alphabet[i] = alphabet[i].strip()

# converts transition function into the form of a dictionary where a tuple of the state and the
input is the key, the power set (as a list) of possible states to go to is the value

transition_function_NFA = list(components_before_start[2].split("}, "))
transition_function_NFA[0] = transition_function_NFA[0].strip("[")

transition_function_dict_NFA = {}

for item in transition_function_NFA:
    item = item.strip()
    item = item.replace("δ", "")

```

```

item = item.replace('"', "")
item = item.replace("(", "")
item = item.replace("}", "")
item = item.replace("{", "")
split_input_output = item.split(" = ")
if len(split_input_output) == 2:
    input_state_input = split_input_output[0].split(", ")
    if len(split_input_output) == 2:
        tuple_state_and_input = tuple(input_state_input)
        list_output_powerset_states = split_input_output[1].split(", ")
        transition_function_dict_NFA[tuple_state_and_input] = list_output_powerset_states
    else:
        print("Invalid format")
else:
    print("Invalid format")

#start state would be the same in DFA assuming there are no spontaneous moves
#(because there would then be no other states accessible from the start state that don't require
consuming an input)

start_state = components_after_transition[0]
start_state = start_state.strip()

accept_states_NFA = components_after_transition[1]

```

```

accept_states_NFA = accept_states_NFA.replace(")", "")
accept_states_NFA = accept_states_NFA.replace("(", "")
accept_states_NFA = accept_states_NFA.replace("}", "")
accept_states_NFA = accept_states_NFA.replace("{", "")
accept_states_NFA = accept_states_NFA.strip()
accept_states_NFA_list = accept_states_NFA.split(" ")
if len(accept_states_NFA_list) > 1:
    for i in range(len(accept_states_NFA_list)):
        accept_states_NFA[i] = accept_states_NFA[i].strip()
        accept_states_NFA[i] = accept_states_NFA[i].strip("{}()")

```

in the form of a list creates powerset of states_NFA (so all subsets of the set of states in NFA)

where each element is a Python tuple representing the subset/state

```

states_DFA = []
states_DFA.append(tuple())
for item in states_NFA:
    states_DFA.append(tuple(item))

```

```

for i in range(1 << len(states_NFA)):
    subset = []
    for j in range(len(states_NFA)):
        if i & (1 << j):
            subset.append(states_NFA[j])
    states_DFA.append(tuple(subset))
states_DFA = list(set(states_DFA))
states_DFA.sort()

# creates dictionary of transitions in DFA form  $Q \times \Sigma \rightarrow Q$  where  $Q$  is a state of the DFA
# (where states are subsets) and every input has an arrow

# the keys are the input (as tuples) and the values are the output (a single tuple element of
states_DFA)

# empty set state points to itself

transition_function_dict_DFA = {}

for symbol in alphabet:
    transition_function_dict_DFA[(states_DFA[0], symbol)] = states_DFA[0]

for state in states_DFA:
    for symbol in alphabet:
        list_accepted_from_multiple_state = []
        for item in state:
            if (item, symbol) in transition_function_dict_NFA:

```

```

output_string = str(transition_function_dict_NFA[(item, symbol)])

final_output_string = output_string.replace('[', '').replace(']', '').replace('""', '')

final_output_string = final_output_string.strip('')

integers = [int(substring) for substring in final_output_string.split(',')]

list_accepted_from_multiple_state.extend(integers)

list_accepted_from_multiple_state = sorted(set(list_accepted_from_multiple_state))

transition_function_dict_DFA[(state, symbol)] =
tuple(list_accepted_from_multiple_state)

# creates list of accept states for DFA by checking if a state's subset contains and accept state
and appends that state if it does

# in the form of a list of tuples where each tuple is a state

accept_states_DFA = []

for state in states_DFA:

    for accept_state in accept_states_NFA_list:

        if accept_state in state:

            accept_states_DFA.append(state)

string_states_DFA = ", ".join(map(str, states_DFA))

string_states_DFA = "{" + string_states_DFA

string_states_DFA = string_states_DFA + "}"

```

```

string_alphabet_DFA = ", ".join(alphabet)

string_alphabet_DFA = "{" + string_alphabet_DFA

string_alphabet_DFA = string_alphabet_DFA + "}"

string_transition_DFA = "["

for key in transition_function_dict_DFA:

    string_value = str(transition_function_dict_DFA[key])

    string_value = string_value.replace("(", "{")

    string_value = string_value.replace(")", "}")

    string_transition_DFA = string_transition_DFA + "δ" + str(key) + " = " + string_value + ", "

+ "\n"

string_transition_DFA = string_transition_DFA[:-3]

string_transition_DFA = string_transition_DFA + "]"

string_start_DFA = str(start_state)

string_accept_states_DFA = ", ".join(map(str, accept_states_DFA))

string_accept_states_DFA = "{" + string_accept_states_DFA

string_accept_states_DFA = string_accept_states_DFA + "}"

print("States: " + string_states_DFA + "\n")

print("Alphabet: " + string_alphabet_DFA + "\n")

```

```
print("Transition Function: " + "\n" + string_transition_DFA + "\n")
```

```
print("Start State: " + string_start_DFA + "\n")
```

```
print("Accept States: " + string_accept_states_DFA)
```

```
main()
```

Screen shots of demonstration of running your program on your chosen NFA, including the representation of the output DFA.

```

nicolego@Nicoles-MacBook-Pro-2 ~ % python3 ./cse105projecttask1.py
Please input a NFA by it's formal definition (Q, Σ, δ, q₀, F) and separating commas for each component. Please define the transition function as a Python list of transitions for each combination of state and input (δ(q, a) = P(Q))
Ex. ({q₀, q₁, q₂}, {a, b}, [δ(q₀, a) = {q₁, q₂}, δ(q₁, b) = {q₂}], q₀, {q₁, q₂}):
({0, 1, 2}, {a, b}, [δ(0, a) = {0, 1}, δ(0, b) = {0}, δ(1, a) = {1}, δ(1, b) = {1, 2}], 0, {2})
['0', '1', '2']
States: {(0, ('0',)), ('0', '1'), ('0', '1', '2'), ('0', '2'), ('1',), ('1', '2'), ('2',)}

Alphabet: {a, b}

Transition Function:
[δ((), 'a') = {},
 δ((), 'b') = {},
 δ(('0',), 'a') = {0, 1},
 δ(('0',), 'b') = {0},
 δ(('0', '1'), 'a') = {0, 1},
 δ(('0', '1'), 'b') = {0, 1, 2},
 δ(('0', '1', '2'), 'a') = {0, 1},
 δ(('0', '1', '2'), 'b') = {0, 1, 2},
 δ(('0', '2'), 'a') = {0, 1},
 δ(('0', '2'), 'b') = {0},
 δ(('1',), 'a') = {1},
 δ(('1',), 'b') = {1, 2},
 δ(('1', '2'), 'a') = {1},
 δ(('1', '2'), 'b') = {1, 2},
 δ(('2',), 'a') = {},
 δ(('2',), 'b') = {}]

Start State: 0

Accept States: {('0', '1', '2'), ('0', '2'), ('1', '2'), ('2',)}
nicolego@Nicoles-MacBook-Pro-2 ~ %

```